NATIONAL INSTITUTE OF TECHNOLOGY SRINAGAR

PROJECT SPECIFICATION AND REQUIREMENTS REPORT

Digital Signal Encoder and Decoder

ITT 036 - Programming Assignment 1

Data Communication | Autumn 2026

Team Member	Enrollment Number
Bharat Bushan	2024BITE054
Rakesh Choudhary	2024BITE047

Prepared from the project source code, README, requirements file, and assignment specification.

Repository: github.com/1Bharat007/digital-signal-ENcoder-nd-deCODER

Document version 1.1 | 2 November 2026

Document purpose and basis

This report describes the **specific Digital Signal Encoder and Decoder implementation** developed for ITT 036 Programming Assignment 1. It is not a generic communications-system specification. Requirements and implementation details are mapped to the actual Python modules and function names in the project repository.

Accuracy note: The report distinguishes implemented behaviour from recommendations and known gaps. It does not claim that the simulator models a real transmission channel or that every edge case has been verified.

Contents

1. Project summary and problem definition
2. Assignment requirements and project objectives
3. Scope, users, and system boundaries
4. Architecture and end-to-end execution flow
5. Functional requirements and traceability
6. Signal representation and exact coding conventions
7. Line-coding algorithms
8. AMI scrambling: B8ZS and HDB3
9. Waveform decoding requirements
10. Analog conversion: PCM and Delta Modulation
11. Module and dependency specification
12. Input validation, exceptions, and edge cases
13. Verification strategy and current test coverage
14. Assumptions, limitations, risks, and future improvements
15. Installation, execution, and acceptance criteria
16. Glossary and references

1. Project summary and problem definition

1.1 Project summary

The Digital Signal Encoder and Decoder is a Python command-line simulation that takes digital bits or sampled analog amplitudes and demonstrates how information can be represented as discrete signal levels. In digital mode, the application generates binary data, finds its longest contiguous palindrome, encodes it with one of five line codes, optionally applies B8ZS or HDB3 to AMI, plots the resulting waveform, and can decode the waveform back into bits. In analog mode, the application converts user-entered samples using PCM or one-bit Delta Modulation, reconstructs an approximation, and then allows the generated bitstream to be passed through the selected line code.

1.2 Problem being addressed

Line coding maps binary information to electrical signal levels or transitions. No single line code is ideal in every respect: some schemes can have long intervals without transitions, some rely on transition timing, and AMI can produce long zero-level runs. Scrambling schemes deliberately insert controlled bipolar violations so that long zero runs contain useful transitions while remaining decodable. Separately, analog-to-digital conversion turns sampled amplitudes into bits using quantization (PCM) or direction decisions (DM).

The educational problem is that these ideas are often studied through separate diagrams and numerical examples. The project unifies them in one executable program and makes the signal patterns visible as plots.

1.3 Main outcomes

- **Digital source:** random bits or random bits with an inserted run of four or eight zeros.
- **Data analysis:** longest contiguous palindromic substring using Manacher's algorithm.
- **Line coding:** NRZ-L, NRZ-I, Manchester, Differential Manchester, and AMI.
- **Scrambling:** AMI with B8ZS or HDB3 substitution.
- **Decoding:** waveform-to-bit decoders for the implemented schemes.
- **Analog conversion:** uniform PCM and one-bit Delta Modulation, both with decoders.
- **Visualization:** discrete voltage waveforms and analog reconstruction staircases.

1.4 Project identity

Attribute	Project-specific value
Course	ITT 036 - Data Communication
Assignment	Programming Assignment 1
Academic term	Autumn 2026
Implementation language	Python 3
Execution model	Local command-line application
Primary dependencies	NumPy and Matplotlib
Authors	Bharat Bushan (2024BITE054); Rakesh Choudhary (2024BITE047)
Repository	https://github.com/1Bharat007/digital-signal-ENcoder-nd-deCODER

2. Assignment requirements and project objectives

The assignment brief asks for a digital data generator, longest-palindrome output, selected line codes, AMI scrambling, and either PCM or DM encoding/decoding. It identifies line decoding as an additional-credit feature. The implementation in the repository contains both PCM and DM, as well as decoders for the listed line codes.

Assignment requirement	Implemented project response
Generate a completely random binary sequence.	`generate_random_bits(length)` uses Python random to generate each bit as 0 or 1.
Generate data with a fixed run of four or eight consecutive zeros.	`generate_bits_with_zero_run(length, run_length)` creates random bits and overwrites a randomly selected window with zeros.

Assignment requirement	Implemented project response
Return the longest palindrome in the generated data.	`longest_palindrome(bits)` uses Manacher's algorithm and returns the longest contiguous palindromic substring.
Implement NRZ-L, NRZ-I, Manchester, Differential Manchester, and AMI.	`line_coding.py` exposes one encoder function for each scheme.
Implement B8ZS and HDB3 scrambling.	`scrambling.py` replaces qualifying zero runs in an AMI-style level sequence.
Implement PCM or DM encoder and decoder.	`analog_modulation.py` implements both PCM and one-bit DM encoder/decoder pairs.
Extra credit: implement line-code decoding.	`line_decoder.py` decodes waveform samples for all five basic schemes and both scrambled AMI schemes.
Visualize the generated/encoded signals.	`main.py` uses Matplotlib step plots for line waveforms and analog reconstruction.

2.1 Project objectives

1. Demonstrate the mapping between a binary sequence and a sampled voltage waveform.
2. Compare how different line codes represent the same bitstream.
3. Show why AMI scrambling is needed for long runs of zeros.
4. Demonstrate receiver-side recovery of bits from an ideal encoded waveform.
5. Demonstrate how sampled analog values become bits through PCM and DM.
6. Keep algorithms modular so each encoder, decoder, and conversion routine can be tested independently.

3. Scope, users, and system boundaries

3.1 Included in the project

- Local generation of digital sequences.
- Insertion of a forced run of four/eight zeros for scrambling demonstrations.
- Longest contiguous palindrome analysis of the current bitstream.
- Ideal line coding and AMI scrambling.
- Ideal waveform decoding using known sample boundaries and initial-state conventions.
- PCM and DM encoding/reconstruction for user-entered analog sample values.
- Command-line interaction, result printing, and Matplotlib plots.
- A small `unittest` suite for selected algorithmic checks.

3.2 Outside the project boundary

- No physical cable, radio, modem, or network hardware is controlled.
- No additive noise, attenuation, jitter, inter-symbol interference, or multipath channel is simulated.
- No clock recovery, bit synchronization, carrier recovery, equalization, or adaptive timing is implemented.
- No real-time streaming or network protocol framing is performed.
- No error correction is claimed for arbitrary corrupted input; the decoders are ideal waveform interpreters, not a full noisy-channel receiver.

3.3 Intended users

The intended users are Data Communication students and instructors. The application is designed for controlled classroom-sized input sequences so that each encoding rule can be followed and verified visually.

3.4 System boundary

```
User input / random generator
|
v
Digital bits <---- OR ----> Analog samples
| |
| PCM encoder / DM encoder
| |
+--------- bitstream --+
|
v
Line-code selection
|
AMI? -- optional B8ZS / HDB3
|
v
Voltage samples
|
Matplotlib plot
|
Optional line decoder
|
Recovered bitstream
|
Analog decoder when analog mode
```

4. Architecture and end-to-end execution flow

4.1 Module architecture

File	Primary responsibility	Important functions / objects
`data_generator.py`	Digital source generation and palindrome analysis.	`generate_random_bits`, `generate_bits_with_zero_run`, `longest_palindrome`
`line_coding.py`	Basic digital line encoders.	`nrz_l`, `nrz_i`, `manchester`, `differential_manchester`, `ami`, `_validate_bits`
`scrambling.py`	AMI-based B8ZS/HDB3 encoders.	`b8zs`, `hdb3`, `_validate_bits`, `_expand`
`line_decoder.py`	Waveform sample validation and decoding.	`decode_nrz_l`, `decode_nrz_i`, `decode_manchester`, `decode_differential_manchester`, `decode_ami`, `decode_b8zs`, `decode_hdb3`
`analog_modulation.py`	PCM and one-bit DM.	`pcm_encode`, `pcm_decode`, `delta_encode`, `delta_decode`, `_validate_samples`
`main.py`	Application orchestration and plotting.	`ENCODERS`, `show_line_signal`, `show_analog_reconstruction`, `main`
`test_project.py`	Automated checks.	`ProjectTests` with five test methods
`requirements.txt`	Third-party dependencies.	`numpy`, `matplotlib`

4.2 Digital input workflow

1. The user selects digital input and enters the desired bit count.
2. The user chooses fully random bits or a forced run of four/eight zeros.
3. The generated bitstream is printed, and `longest_palindrome` is called.
4. The user selects one of the five basic line-coding schemes.
5. If AMI is selected, the user may keep ordinary AMI or choose B8ZS/HDB3.
6. The selected encoder returns a list of voltage samples; the samples are printed and plotted.
7. The user may request decoding. The corresponding decoder reconstructs bits and the program compares them with the input.

4.3 Analog input workflow

1. The user enters a comma-separated list of analog sample amplitudes.
2. The user selects PCM and a bit depth, or DM and a positive step size.
3. The selected analog encoder produces a digital bitstream. PCM also returns quantizer metadata.
4. The matching analog decoder creates a reconstructed amplitude staircase, which is plotted beside the input samples.
5. The generated bits are then processed by the selected line-coding scheme (and optional AMI scrambling).
6. If line decoding is requested, the recovered bits are compared with the analog encoder output. If they match, the analog decoder is run again to display the final reconstruction.

5. Functional requirements and traceability

ID	Functional requirement	Implementation location
FR-01	Generate a non-empty random binary sequence of requested length.	`data_generator.generate_random_bits`
FR-02	Insert exactly a selected 4- or 8-zero window at a random valid start index in a random sequence.	`data_generator.generate_bits_with_zero_run`
FR-03	Return the longest contiguous palindrome in a bit sequence.	`data_generator.longest_palindrome`
FR-04	Encode binary input using the documented NRZ-L convention.	`line_coding.nrz_l`
FR-05	Encode binary input using NRZ-I, with initial level +1 and a transition for bit 1.	`line_coding.nrz_i`
FR-06	Encode binary input using the project's Manchester convention.	`line_coding.manchester`
FR-07	Encode binary input using Differential Manchester with the project initial state.	`line_coding.differential_manchester`
FR-08	Encode binary input using AMI with alternating mark polarity.	`line_coding.ami`
FR-09	Replace each eligible 8-zero sequence using B8ZS.	`scrambling.b8zs`
FR-10	Replace each eligible 4-zero sequence using HDB3 parity rules.	`scrambling.hdb3`
FR-11	Decode supported waveforms into the original bitstream under ideal assumptions.	`line_decoder.py`
FR-12	Quantize analog samples into PCM bit groups and reconstruct quantizer midpoint values.	`analog_modulation.pcm_encode`, `pcm_decode`
FR-13	Encode analog samples into one-bit up/down decisions and reconstruct a staircase.	`analog_modulation.delta_encode`, `delta_decode`
FR-14	Plot the generated signal and analog reconstruction.	`main.show_line_signal`, `main.show_analog_reconstruction`
FR-15	Provide an interactive command-line execution path.	`main.main` and `if __name__ == "__main__"`
FR-16	Provide automated unit tests for selected functions.	`test_project.py`

5.1 Non-functional requirements

ID	Category	Project-specific requirement
NFR-01	Usability	Prompts identify digital/analog mode, coding choice, scrambling choice, and decode option.
NFR-02	Modularity	Generation, coding, scrambling, decoding, analog conversion, and orchestration are split across files.
NFR-03	Correctness	Encoder and decoder conventions and their initial states must match.
NFR-04	Performance	The core processing algorithms should be linear in the input length for ordinary list-based inputs.
NFR-05	Maintainability	Validation and signal conventions should be documented and reused consistently.
NFR-06	Portability	Python 3.10+ with the two listed third-party packages is the documented runtime.
NFR-07	Testability	Core functions can be imported and tested independently of the CLI.
NFR-08	Visualization	Plots label voltage levels and bit/sample axes to support interpretation.

6. Signal representation and exact coding conventions

6.1 Voltage alphabet and sampling model

The project uses the discrete voltage alphabet **{-1, 0, +1}**. Each logical bit is represented by **two samples**, one for each half of the bit interval. This is why an input of n bits produces 2n voltage samples. The two-sample representation is essential for Manchester-family schemes, where the middle-of-bit transition is the data-bearing feature.

A bit is the information symbol (0 or 1). A sample is a voltage level at one discrete half-bit interval. Do not describe the two samples as two separate bits.

6.2 Exact conventions implemented

Scheme	Project convention	Initial state
NRZ-L	0 = +1; 1 = -1. The level stays constant through both samples of a bit.	No history needed.
NRZ-I	Bit 1 toggles the level; bit 0 leaves it unchanged.	Level starts at +1.
Manchester	0 = [+1, -1] (high-to-low); 1 = [-1, +1] (low-to-high).	No previous bit state.
Differential Manchester	A 0 causes a transition at the beginning; every bit has a mandatory middle transition.	Level starts at +1.
AMI	0 = [0, 0]; each 1 is a pulse whose sign alternates.	Previous-pulse variable starts at -1, so first 1 is +1.

6.3 Concrete example

For input bits **[0, 1]**, the expected sample arrays are:

Scheme	Two-bit output samples
NRZ-L	[+1, +1, -1, -1]
NRZ-I	[+1, +1, -1, -1]
Manchester	[+1, -1, -1, +1]
Differential Manchester	[-1, +1, +1, -1]
AMI	[0, 0, +1, +1]

The NRZ-L and NRZ-I outputs happen to match for this particular two-bit input under the selected initial state; they are not the same rule. A different input sequence exposes their difference.

7. Line-coding algorithms

7.1 NRZ-L

The encoder converts each bit directly to a level: +1 for 0 and -1 for 1. The selected level is repeated twice so both half-bit samples are equal. The decoder groups samples in pairs, verifies that the two samples match, and maps +1 back to 0 and -1 back to 1.

7.2 NRZ-I

NRZ-I uses the presence or absence of a transition rather than an absolute voltage value to represent the bit. The encoder starts at +1. For each 1 it multiplies the current level by -1; for each 0 it leaves the level unchanged. The new level is repeated twice. The decoder compares each bit-level with the previous level: a change means 1, no change means 0. The decoder must use the same initial level (+1).

7.3 Manchester

Manchester always changes level in the middle of the bit. In this project, 0 is [+1, -1] and 1 is [-1, +1]. The compulsory middle transition provides a timing transition in every bit, while direction determines the data value. The decoder reads each pair and accepts only these two valid patterns.

7.4 Differential Manchester

Differential Manchester always has a middle transition. The presence of a transition at the start of the bit determines the data: in this implementation, a start transition represents 0 and no start transition represents 1. The encoder begins with level +1. For a 0, it toggles before appending the first half; it then toggles again for the mandatory middle transition. The decoder compares the first half-level with the previous bit's ending level, then remembers the current second half as the previous level for the next bit.

7.5 AMI

Alternate Mark Inversion represents 0 as zero voltage and each 1 as a non-zero pulse. Successive 1 pulses alternate signs. The implementation initializes `pulse = -1` and toggles before appending a 1, so the first mark is +1. A 0 leaves the pulse polarity state unchanged. The returned level for each bit is duplicated twice.

7.6 Complexity

Each encoder performs one pass over n bits and emits a constant number of samples per bit. Therefore the expected time complexity is O(n) and output storage is O(n). The stateful encoders use O(1) additional state apart from the output list.

8. AMI scrambling: B8ZS and HDB3

8.1 Why scrambling is needed

In AMI, a 0 is transmitted as zero voltage. A long run of zeros can therefore produce a long interval with no transitions. This can make receiver timing difficult. B8ZS and HDB3 replace selected zero runs with controlled bipolar patterns containing deliberate violations of the normal AMI alternation rule. The decoder recognizes these special patterns and restores the original zeros.

8.2 B8ZS as implemented

The encoder looks for exactly eight consecutive zeros in the next eight input positions. Let p be the polarity of the last AMI pulse. The replacement is:

```
0, 0, 0, p, -p, 0, -p, p
```

The non-zero pulses at the violation locations deliberately break ordinary AMI alternation. If p = +1, the replacement becomes `0, 0, 0, +1, -1, 0, -1, +1`. If p = -1, all signs reverse. The implementation advances by eight input bits after substitution and sets the remembered last pulse to the final replacement level.

8.3 HDB3 as implemented

HDB3 replaces a run of four zeros. The function tracks the number of non-zero AMI marks since the previous violation. It selects one of two patterns:

Parity of marks since violation	Pattern	Rule
Even	B00V	B is the next normal AMI polarity (opposite the preceding pulse); V repeats B, creating a violation.
Odd	000V	The first three positions remain zero; V repeats the preceding non-zero pulse polarity.

After inserting either pattern, the marks-since-violation counter is reset. Ordinary input 1 bits continue alternating AMI polarity and increase the count; input zero bits add zero level without increasing the count.

8.4 Important implementation assumptions

- Both scramblers initialize the remembered previous pulse to -1, matching the AMI encoder convention in this project.
- The functions process original logical bits and produce the scrambled signal levels directly; they do not call the basic `ami()` function internally.
- The scrambler returns two samples per encoded bit by expanding every generated level twice.
- The decoders use the same initial previous-pulse assumption and pattern conventions.

9. Waveform decoding requirements

The decoder accepts voltage samples, not the original bit list. It first validates that the signal is non-empty, contains an even number of samples, and uses only -1, 0, or +1 after integer conversion. For NRZ-style and AMI signals, `_get_levels` groups each two-sample bit interval and checks that both samples are equal. Manchester-family decoders instead inspect the two half-bit samples directly.

Decoder	How a bit is recovered
`decode_nrz_l`	Read each constant pair: +1 means 0; -1 means 1.
`decode_nrz_i`	Compare the current level with the previous level: transition = 1; no transition = 0.
`decode_manchester`	[-1, +1] means 1; [+1, -1] means 0; any other pair is rejected.
`decode_differential_manchester`	A transition from the previous ending level to the first half means 0; no start transition means 1. The middle transition is required.
`decode_ami`	Zero level means 0; either non-zero polarity means 1.
`decode_b8zs`	Recognize the exact eight-level replacement pattern relative to last-pulse polarity; emit eight zeros. Otherwise decode AMI levels.
`decode_hdb3`	Recognize `000V` or `B00V` relative to remembered pulse polarity; emit four zeros. Otherwise decode ordinary AMI levels.

9.1 Decoder boundary

These are ideal, sample-aligned decoders. They assume the two samples for each bit are already correctly segmented. They do not estimate timing, repair corrupted samples, or perform clock recovery. A malformed Manchester pair or missing Differential Manchester midpoint transition produces a `ValueError` instead of a decoded bit.

10. Analog conversion: PCM and Delta Modulation

10.1 Shared sample validation

`_validate_samples` converts every input sample to a Python float, rejects an empty list, and checks that every value is finite using `math.isfinite`. This prevents NaN and infinity from entering the quantization or comparison logic.

10.2 Pulse Code Modulation (PCM)

The project uses uniform mid-rise quantization over the range of the entered sample values. For b bits per sample, the number of quantization levels is L = 2^b. The range is divided into equal intervals of width Δ = (vmax - vmin) / L. Each sample is mapped to an integer bin index and serialized as a zero-padded b-bit binary word.

```
L = 2 ** bits_per_sample
step = (vmax - vmin) / L
index = int((sample - vmin) / step)
index = clamp(index, 0, L - 1)
```

The encoder returns both the bitstream and metadata containing `bits_per_sample`, `vmin`, `vmax`, and `step`. The metadata is required in this simulation because the decoder needs the quantizer range and bit grouping. The decoder converts each b-bit group back to its index and reconstructs the midpoint of that quantization interval:

```
reconstructed = vmin + (index + 0.5) * step
```

If all input samples have the same value, the encoder expands the range by a small padding amount so the quantization step is not zero. Bit depth is restricted by the encoder to 1 through 16 bits per sample.

10.3 Delta Modulation (DM)

The DM encoder stores a running estimate, initially 0.0 by default. For each input sample, it compares the sample with the current estimate. If the sample is greater than or equal to the estimate, it emits 1 and increases the estimate by the step size. Otherwise it emits 0 and decreases the estimate by the step size. The decoder performs the same accumulation: 1 means step up; 0 means step down.

```
if sample >= estimate:
bit = 1
estimate += step_size
else:
bit = 0
estimate -= step_size
```

The step size must be positive. The encoder and decoder must use the same step size and initial reconstruction value. A large step may cause granular noise; a small step may fail to follow a rapidly changing signal (slope overload).

10.4 PCM versus DM in this project

Property	PCM implementation	DM implementation
Information per analog sample	`bits_per_sample` bits	1 bit
Decision	Choose quantization interval for absolute sample amplitude.	Choose direction relative to running estimate.
Decoder state	Range, bit depth, and group boundaries.	Previous reconstructed estimate and step size.
Reconstruction	Midpoint of selected quantization interval.	Accumulated staircase of fixed-size up/down steps.
Trade-off	More bits per sample can improve amplitude resolution.	Performance depends strongly on step size and signal slope.

11. Module and dependency specification

11.1 Python runtime

The README specifies Python 3.10 or newer. This is consistent with the use of built-in generic type annotations such as `list[int]`. The program is run from the repository directory with `python main.py`.

11.2 Third-party packages

Package	Where it is used	Why it is present
NumPy (`numpy`)	Imported in `main.py` as `np`. Used by `show_line_signal` for integer-array conversion, sample/time coordinate generation, and appending the final step-plot value.	Convenient numerical arrays and coordinate construction for plots. The algorithmic encoders themselves primarily use Python lists.
Matplotlib (`matplotlib`)	Imported as `matplotlib.pyplot` in `main.py`. Used in both plotting functions.	Creates the line waveform step plot and analog input/reconstruction plot.

11.3 Python standard library

Module / feature	Use in the project
`random`	Creates random binary bits and chooses the random start position for a forced zero run.
`math`	Checks whether analog sample values are finite.
`unittest`	Provides test cases and assertions in `test_project.py`.
`format(value, "0Nb")`	Built-in binary formatting used to serialize a PCM index into a fixed-width bit group.
Lists, slicing, loops, dictionaries	Represent bitstreams and voltage samples; detect patterns; hold encoder/decoder mappings in `ENCODERS`.

11.4 Dependency installation

```
python -m pip install -r requirements.txt
python main.py
python -m unittest -v
```

The current `requirements.txt` lists only `numpy` and `matplotlib`. `random`, `math`, and `unittest` are part of Python and do not need separate pip installation.

12. Input validation, exceptions, and edge cases

Area	Implemented validation	Cases to verify or strengthen
Binary generators	Length must be at least 1; forced run must be 4 or 8; total length must fit the run.	Non-integer length values from CLI and reproducibility via optional random seed.
Line encoders	Reject empty inputs and values outside 0/1.	Test bools or non-integer numeric inputs if strict type enforcement is desired.
Scramblers	Reject empty/non-binary input.	Runs at beginning/end, adjacent qualifying runs, and both preceding pulse polarities.
Line decoders	Require non-empty even sample count and levels in {-1,0,+1}; `_get_levels` checks constant pairs.	`_check_signal` converts to int before checking; fractional values can be truncated, so strict pre-conversion validation would be safer.
PCM encoder	Requires finite non-empty samples and 1-16 bits per sample. Handles constant-valued input with range padding.	PCM decoder should also validate positive bit depth, range order, and metadata consistency.
DM	Requires finite non-empty samples and positive step size; decoder checks non-empty binary input and positive step.	Very large/small step sizes and rapid signal changes.
CLI	Rejects unknown menu choices using `ValueError`.	Non-numeric text supplied to `int()` or `float()` is not wrapped in a friendly retry loop.

12.1 Required edge-case test vectors

- Empty sequence and single-bit sequence.
- All-zero and all-one bitstreams.
- Four zeros and eight zeros at the start, middle, and end.
- Consecutive or overlapping candidate zero windows.
- AMI mark polarity beginning with the project's initial previous pulse = -1.
- Both even and odd HDB3 mark counts before a four-zero run.
- Manchester invalid pair and Differential Manchester missing midpoint transition.
- PCM input with all equal values and values at both range endpoints.
- PCM bitstream whose length is not divisible by the selected bit depth.
- DM ramp and rapidly changing input to demonstrate step-size limits.

13. Verification strategy and current test coverage

13.1 Existing automated tests

Test method	Current behaviour checked	Coverage limitation
`test_basic_line_codes`	Round-trip for NRZ-L, NRZ-I, Manchester, Differential Manchester, and AMI using `[1,0,1,1,0,0,1,0]`.	One main reference bitstream; does not exhaustively test all short bit patterns or invalid samples.
`test_b8zs`	Round-trip for three hand-written bitstreams, including an eight-zero run.	Does not exhaustively test both signs and all surrounding contexts.
`test_hdb3`	Round-trip for three hand-written bitstreams covering selected four-zero runs.	More parity states and pulse polarities should be added.
`test_analog`	Runs PCM and DM on five samples and checks reconstructed output lengths.	Does not check PCM numerical error, exact PCM levels, DM bit decisions, or DM reconstruction error.
`test_palindrome`	Checks one example whose longest palindrome is `[1,0,0,1]`.	Does not test even/odd ties, single-bit input, all identical bits, or empty input.

13.2 Recommended verification matrix

Test class	Expected assertion
Round-trip line code	For valid bits and matching initial state: `decode(encode(bits)) == bits`.
Scrambler round trip	For every test vector: `decode_b8zs(b8zs(bits)) == bits` or `decode_hdb3(hdb3(bits)) == bits`.
PCM accuracy	Each reconstructed value should lie at the midpoint of its selected bin; error should be bounded by half a quantization step for in-range samples.
DM behaviour	Check exact output bits for a known sample sequence and verify the decoder's cumulative +/- step staircase.
Palindrome correctness	Compare against known longest contiguous palindrome results for odd, even, tied, one-element, and all-equal cases.
Validation	Invalid lengths, symbols, sample counts, bit depths, and step sizes should produce clear exceptions.

Test status: This report describes the tests present in `test_project.py`; it does not assert that the suite was executed during report preparation. Run `python -m unittest -v` on the final submission and record the actual result.

14. Assumptions, limitations, risks, and future improvements

14.1 Current assumptions

- Voltage levels are ideal normalized values -1, 0, and +1.
- Two samples correspond to each bit interval.
- The decoder knows the bit boundaries and the same initial state as the encoder.
- NRZ-I and Differential Manchester start from +1.
- AMI/B8ZS/HDB3 previous-pulse state starts at -1.
- PCM range metadata is available to the decoder; it is not transmitted as an in-band protocol header.
- DM encoder and decoder share the step size and initial reconstruction value 0.
- No channel noise or physical distortion is introduced.

14.2 Known limitations

- The application simulates ideal source-to-encoder-to-decoder processing, not a full communication channel.
- There is no timing recovery; the decoder receives already aligned two-sample bit groups.
- PCM metadata is passed in program memory for decoding, not serialized as a separate transmission header.
- The PCM tests check output length rather than a numeric quality metric.
- The DM tests check output length rather than exact bit decisions or signal error.
- The current test suite is compact and does not exhaustively validate generator and input-validation edge cases.
- The command-line interface terminates on several malformed inputs instead of prompting the user again.
- The repository currently includes a `__pycache__` directory; generated Python cache files should normally be excluded with `.gitignore`.

14.3 Recommended future improvements

1. Add parameterized deterministic test vectors for all line codes, both scrambler polarities, and HDB3 parity states.
2. Add numerical PCM error metrics such as mean squared error and maximum absolute error.
3. Add DM experiments that compare several step sizes and illustrate granular noise and slope overload.
4. Add optional channel noise, amplitude scaling, and sample corruption to test decoder robustness.
5. Add timing-offset demonstrations and a separate synchronization stage.
6. Strengthen decoder input validation before integer conversion and validate PCM decoder parameters.
7. Add friendly CLI input loops and clearer error messages.
8. Add a random seed option so demonstrations can be reproduced exactly.

9 Add a project `.gitignore` for `__pycache__/`, `*.py[cod]`, and virtual environments.

15. Installation, execution, and acceptance criteria

15.1 Environment setup

```
# 1. Open a terminal in the cloned project directory
python --version

# 2. Install the declared dependencies
python -m pip install -r requirements.txt

# 3. Run the interactive application
python main.py

# 4. Run automated tests
python -m unittest -v
```

15.2 Acceptance criteria

- The documented Python runtime and required packages install successfully.
- `main.py` launches and presents digital and analog input paths.
- Random bit generation returns the requested number of valid bits.
- Forced zero-run generation inserts a requested run length of four or eight.
- The palindrome function returns a longest contiguous binary palindrome for its input.
- All five basic encoders produce exactly two samples per input bit.
- B8ZS and HDB3 preserve bit count through substitution and have matching decoders for valid supported patterns.
- PCM creates fixed-width code groups and reconstructs one value per input sample with the saved metadata.
- DM emits one decision bit per input sample and reconstructs one staircase value per bit.
- Plots show signal levels and reconstruction values with meaningful labels.
- Tests are executed on the final version and their actual result is recorded.

15.3 Submission checklist

- Both authors and enrollment numbers are correct on the cover page.
- The final GitHub branch, README, requirements file, and source modules match the submitted version.
- The full test command is run after the final code change.
- Generated cache folders and virtual environments are excluded from the repository.
- The report does not claim hardware transmission, noise simulation, synchronization, or error correction that is not implemented.
- Any use of external references, discussions, or AI assistance is acknowledged according to the instructor's rules.

16. Glossary and references

Term	Meaning in this project
Bit	One binary information symbol: 0 or 1.
Bit interval	The duration allocated to one bit; represented here by two half-bit samples.
Line coding	Mapping binary bits to signal voltage levels or transitions.
AMI	Alternate Mark Inversion; zeros use zero level and successive ones alternate polarity.
Bipolar violation	A non-zero pulse that intentionally breaks the expected AMI alternation rule.
Scrambling	A pattern substitution that improves transition density while allowing the receiver to recover the original zeros.
PCM	Pulse Code Modulation; encodes quantized sample amplitudes.
DM	Delta Modulation; encodes the direction of change in a running reconstruction.
Quantization step	The amplitude width represented by one PCM interval.
Round-trip test	Encoding and then decoding a source sequence and comparing the recovered result.
Palindrome	A contiguous sequence that reads the same forward and backward.

References

- Project repository and source files: https://github.com/1Bharat007/digital-signal-ENcoder-nd-deCODER
- Project README and signal conventions: https://github.com/1Bharat007/digital-signal-ENcoder-nd-deCODER/blob/master/README.md
- Assignment-specific project specification: `specification_report.md` in the repository and the ITT 036 Programming Assignment 1 handout supplied for the course.
- Python documentation: https://docs.python.org/3/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/stable/
- Manacher, G. (1975). A new linear-time “on-line” algorithm for finding the smallest initial palindrome of a string. Journal of the ACM, 22(3), 346–351.

Final document note: The report is written from the public repository contents inspected on 2 November 2026. Before formal submission, compare it with the instructor's latest assignment wording and add any required course-specific cover-page fields.